

유니티 팀 프로젝트 - 벙커 생활 시뮬레이션



기술 경험

- AI 행동트리
- A* 길찾기 알고리즘
- 디자인패턴, 아키텍처 패턴
- 형상 관리 방법(Fork)
- GPU 최적화(GPU 인스턴싱, 오클루전 컬링)
- 맵 에디터

Project LUP - bunker 생활 시뮬레이션

Project LUP(Level UP)는 **13명의 프로그래머 중 2~3인이 게임 장르별로 팀을 구성하고, 공동의 프레임워크와 한정된 리소스 내에서 자신의 실력을 한층 성장 시키는 것을 목표로 하는 프로젝트** 입니다.

사용 기술 : Unity3D, JSON

장르 : SLG (전략 시뮬레이션)

플랫폼 : Android

엔진 버전 : Unity 6000.6.2f1

참여 인원 : 개발(13)

협업 도구 : Fork, Discord, Notion, Github

GitHub : <https://github.com/a-i-am/projectlup-v2025-personal>



작업자를 이용해 bunker 시설을 관리하는 전략 시뮬레이션 게임

저는 SLG 프로젝트를 담당한 팀에서

작업자 행동을 컨트롤하는 작업자 AI 시스템을 담당하였습니다.

핵심 구현 기능

- AI 행동트리
- A* 길찾기 알고리즘



대기실 주변 배회



작업 중

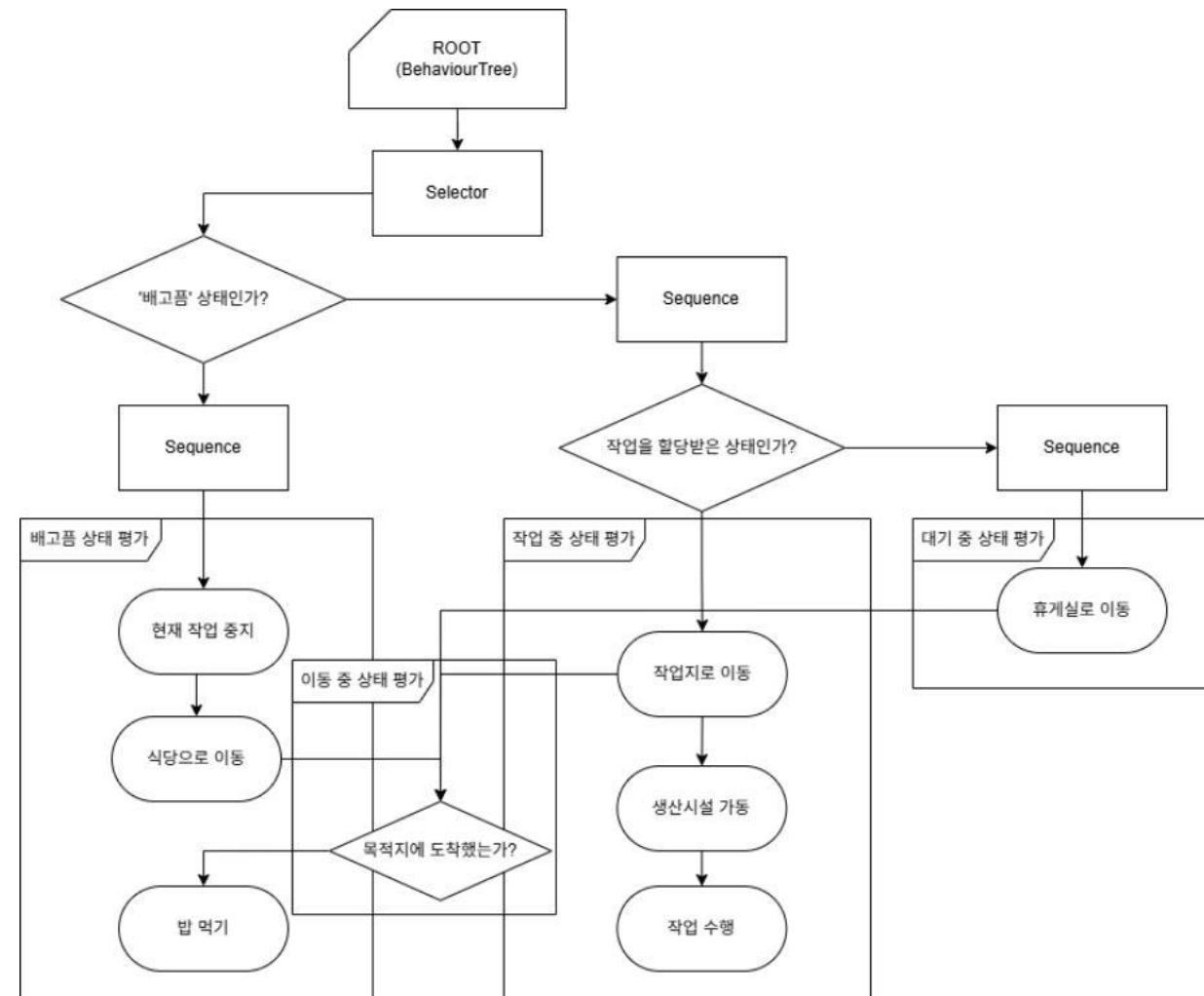


식당으로 이동

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

AI 행동 트리 : 작업자 행동 우선순위 평가

'배고프면 식당으로 이동', '작업 요청된 작업지로 이동' 등 작업자의 AI 행동흐름을 구현했습니다.

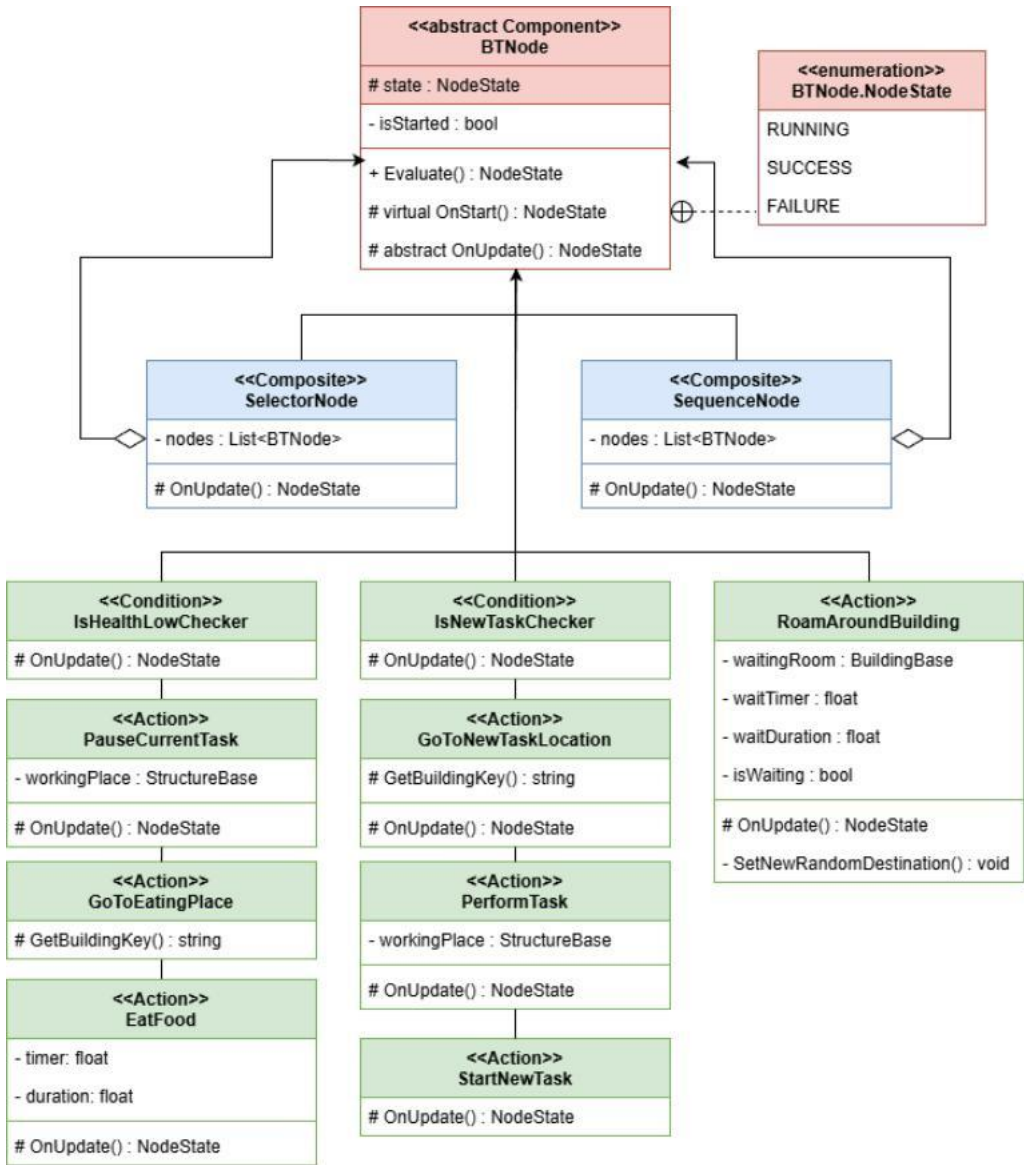


- FSM은 상태 전이가 깊어질 때 스택에 너무 많은 상태가 쌓이면 메모리 오버플로우 위험이 있었습니다.
- HFSM은 하위 상태들이 스스로 다음 행동을 결정하기 때문에 전체 상태의 우선순위를 평가하기엔 부적합할 것 같았습니다
- 따라서 하위 상태의 우선순위를 평가한 후에 역할을 배정하는 구조인 행동 트리(Behaviour Tree)를 선택했습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

AI 행동 트리 : 디자인 패턴을 활용한 객체지향적 설계

(1) 컴포지트 패턴



(Github) BTreeNode.cs

[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/BTreeNode.cs at feature/PCR/PSE_Current · project-lup/projectlup-v2025](#)

- **Component (BTreeNode)** : 모든 노드의 공통 인터페이스
- **Composite (Sequence, Selector)** : 자식을 가지는 노드
 - **Selector** : 여러 행동 중 하나라도 성공하면 즉시 종료하고 성공을 반환하는 노드
 - **Sequence** : 모든 자식 노드를 순서대로 실행해야 성공하는 노드
- **Leaf (Condition, Action)** : 실제 행동을 하는 말단 노드
 - **Condition** : 자식을 가지는 노드
 - **Action** : 자식을 가지는 노드

- 컴포지트 패턴으로 AI의 개별 동작과 이를 제어하는 논리 단위(Composite)를 BTreeNode라는 인터페이스에 상속시켜 추상화하였습니다.
- 덕분에 어떤 노드인지 조건문으로 매번 판단할 필요 없이 모든 노드를 List<BTreeNode> 에 담아 순차적으로 상태를 평가할 수 있게 되었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

AI 행동 트리 : 디자인 패턴을 활용한 객체지향적 설계

(2) 템플릿 메서드 패턴

```
public NodeState Evaluate()
{
    if (!isStarted)
    {
        OnStart();
        isStarted = true;
    }

    state = OnUpdate();

    if (state != NodeState.RUNNING)
    {
        isStarted = false;
    }

    return state;
}
참조 2개
protected virtual void OnStart() { }
참조 13개
protected abstract NodeState OnUpdate();
```

(Github) BTNode.cs

[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/BTNode.cs](https://github.com/projectlup/projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/BTNode.cs)
at feature/PCR/PSE_Current · project-lup/projectlup-v2025

```
protected override BTNode.NodeState OnUpdate()
{
    foreach (BTNode node in nodes)
    {
        switch (node.Evaluate())
        {
            case NodeState.RUNNING:
                return NodeState.RUNNING;
            case NodeState.SUCCESS:
                return NodeState.SUCCESS;
            case NodeState.FAILURE:
                continue;
        }
    }

    return NodeState.FAILURE;
}
```

(Github) SelectorNode.cs

[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/SelectorNode.cs](https://github.com/projectlup/projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/SelectorNode.cs)
at feature/PCR/PSE_Current · project-lup/projectlup-v2025

```
protected override void OnStart()
{
    if (HasData(GetBuildingKey()))
    {
        targetPlace = GetData<BuildingBase>(GetBuildingKey());

        if (targetPlace != null)
        {
            UpdatePath();
        }
    }
}
```

(Github) MoveActionBase.cs

[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/MoveActionBase.cs](https://github.com/projectlup/projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/MoveActionBase.cs)
at feature/PCR/PSE_Current · project-lup/projectlup-v2025

```
protected override NodeState OnUpdate()
{
    float currentHunger = GetData<float>(BBKeys.Hunger);
    bool isHungry = currentHunger >= HungerRules.HungryThreshold;

    OwnerAI.IsHunger = isHungry;

    if (isHungry)
    {
        return ReturnAndLog(NodeState.SUCCESS, "배고픔 감지됨!");
    }
    else
    {
        return ReturnAndLog(NodeState.FAILURE, "아직 배고프지 않음.");
    }
}
```

(Github) IsHealthLowChecker.cs

[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/ConditionNodes/IsHealthLowChecker.cs](https://github.com/projectlup/projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/BT/ConditionNodes/IsHealthLowChecker.cs)
at feature/PCR/PSE_Current · project-lup/projectlup-v2025

- 템플릿 메서드 패턴으로 행동트리 노드의 전체적인 실행 흐름(Lifecycle)을 상위 클래스에 고정하였습니다.
- 덕분에 모든 노드에서 반복되는 상태 관리 로직을 중복 작성할 필요 없게 되었습니다.
- '작업자가 목적지로 이동' 하는 로직은 동일하기 때문에, OnStart()라는 함수로 공통된 초기화 로직은 통일하고 데이터만 다르게 받도록 했습니다.
- OnUpdate()라는 함수로 모든 행동 트리 노드가 '현재 상태의 행동을 평가'하고 현재 노드 상태를 반환하는 로직을 이어받게끔 하였습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

AI 행동 트리 : 디자인 패턴을 활용한 객체지향적 설계

(3) 전략 패턴

<<Client>> WorkerAI	
- worker : Worker - root : BTNode - mover : UnitMover + currentTaskPlace : StructureBase + myInfo : WorkerInfo + Hunger : float <<property>> + IsHunger : bool <<property>> + HasTask : bool <<property>>	
- CheckHungerState() : void + Initialize(info : WorkerInfo, restaurant : BuildingBase, station : BuildingBase) : void + InitBlackboard() : void + InitBTReferences() : void + AssignTask(workingPlace : StructureBase) : void - SettingBT() : void + UpdateBT() : NodeState	

```
private void SettingBT()
{
    BTNode hungerSequence = new SequenceNode(new List<BTNode>
    {
        new IsHealthLowChecker(LocalBlackboard),
        new PauseCurrentTask(LocalBlackboard),
        new GoToEatingPlace(LocalBlackboard),
        new EatFood(LocalBlackboard)
    });

    BTNode workingSequence = new SequenceNode(new List<BTNode>
    {
        new IsNewTaskChecker(LocalBlackboard),
        new GoToNewTaskLocation(LocalBlackboard),
        new StartNewTask(LocalBlackboard),
        new PerformTask(LocalBlackboard)
    });

    root = new SelectorNode(new List<BTNode>
    {
        hungerSequence,
        workingSequence,
        new RoamAroundBuilding(LocalBlackboard)
    });
}
```

(Github) WorkerAI.cs (121번줄 ~ 144번줄)

[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/WorkerAI.cs](https://github.com/projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/WorkerAI.cs)
at feature/PCR/PSE_Current · project-lup/projectlup-v2025

- 캐릭터의 행동이 늘어날수록 작업자의 행동 순서 및 행동 로직을 관리하기 복잡해진다는 문제가 있었습니다.
- 그래서 트리 노드에 속하는 각 행동의 세부 구현을 New List<BTNode> 내부에서 객체로 초기화 되게 하였습니다.
- 그 결과, 행동 노드 객체의 초기화 순서를 제어하게 하여 AI의 행동 우선순위를 유연하게 관리할 수 있게 되었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

AI 행동 트리 : 아키텍처 패턴을 활용한 객체지향적 설계

(4) 블랙보드 패턴

<<Client>> WorkerAI
- worker : Worker - root : BTNode - mover : UnitMover + currentTaskPlace : StructureBase + myInfo : WorkerInfo + Hunger : float <<property>> + IsHunger : bool <<property>> + HasTask : bool <<property>>
- CheckHungerState() : void + Initialize(info : WorkerInfo, restaurant : BuildingBase, station : BuildingBase) : void + InitBlackboard() : void + InitBTReferences() : void + AssignTask(workingPlace : StructureBase) : void - SettingBT() : void + UpdateBT() : NodeState

```
public static class BBKeys
{
    public const string OwnerAI = "OwnerAI";
    public const string Self = "Self";
    public const string UnitMover = "UnitMover";

    public const string Hunger = "Hunger";
    public const string IsHunger = "IsHunger";

    public const string HasTask = "HasTask";

    public const string WorkerStation = "WorkerStation";
    public const string Restaurant = "Restaurant";
    public const string AssignedWorkplace = "AssignedWorkplace";
}
```

(Github) BBKeys.cs

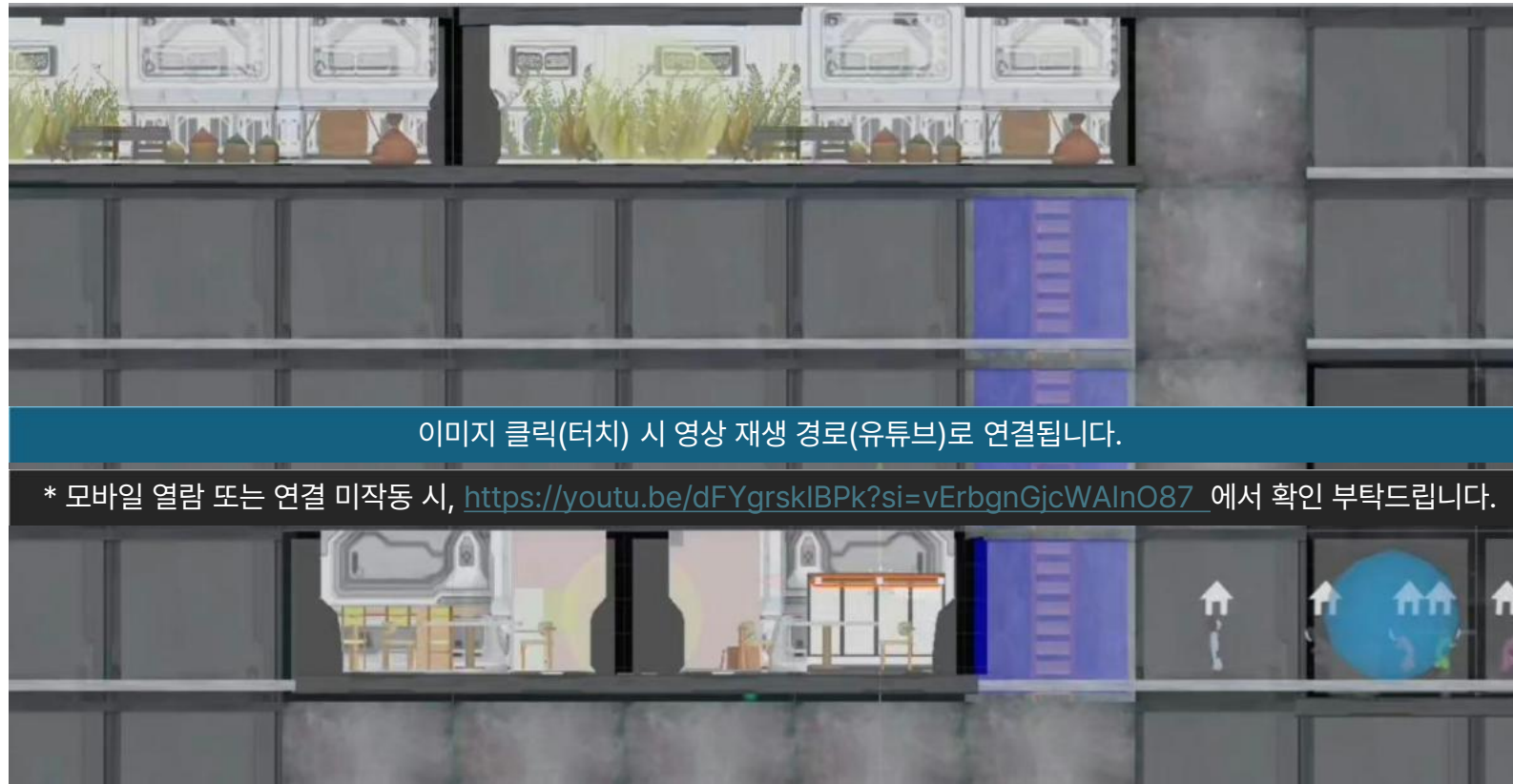
[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/Blackboard/BBKeys.cs](https://github.com/projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/Blackboard/BBKeys.cs)
at feature/PCR/PSE_Current · project-lup/projectlup-v2025

- AI의 행동 패턴이 다양해짐에 따라 여러 로직에서 공통으로 참조해야 하는 데이터가 분산되어, 상태 동기화 및 관리가 복잡해지는 문제가 있었습니다.
- 이에 WorkerBlackboard라는 중앙 데이터 저장소를 두어 모든 상태 데이터를 한곳에서 관리함으로써, 데이터 의존성을 최소화하였습니다.
- 또한 개별 행동 노드들이 데이터를 직접 주고받을 경우 발생할 수 있는 높은 결합도(Coupling)와 유지보수의 어려움을 방지하였습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

A* 길찾기 알고리즘(A star Pathfinding) : 2D 그리드 환경 내 최적 경로 탐색

모든 작업자가 전체 2D 그리드 타일 맵을 가장 빠른 동선으로 돌아다니게 하기 위해 A* 길찾기 알고리즘(A Star Pathfinding)을 적용하였습니다.



(Github) AStar 로직에 사용된 스크립트들

[projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/AStar at feature/PCR/PSE_Current · project-lup/projectlup-v2025](https://github.com/projectlup/projectlup-v2025/Assets/2_Scripts/Games/PCR/6_Worker/AStar%20at%20feature/PCR/PSE_Current)

```
public APathfinding(AGridMap map)
{
    gridMap = map;

    openList = new Heap<ANode>(gridMap.grid.Length);
    closedSet = new HashSet<ANode>();
    cachedPath = new List<ANode>();
    cachedNeighbors = new List<ANode>(4);
}

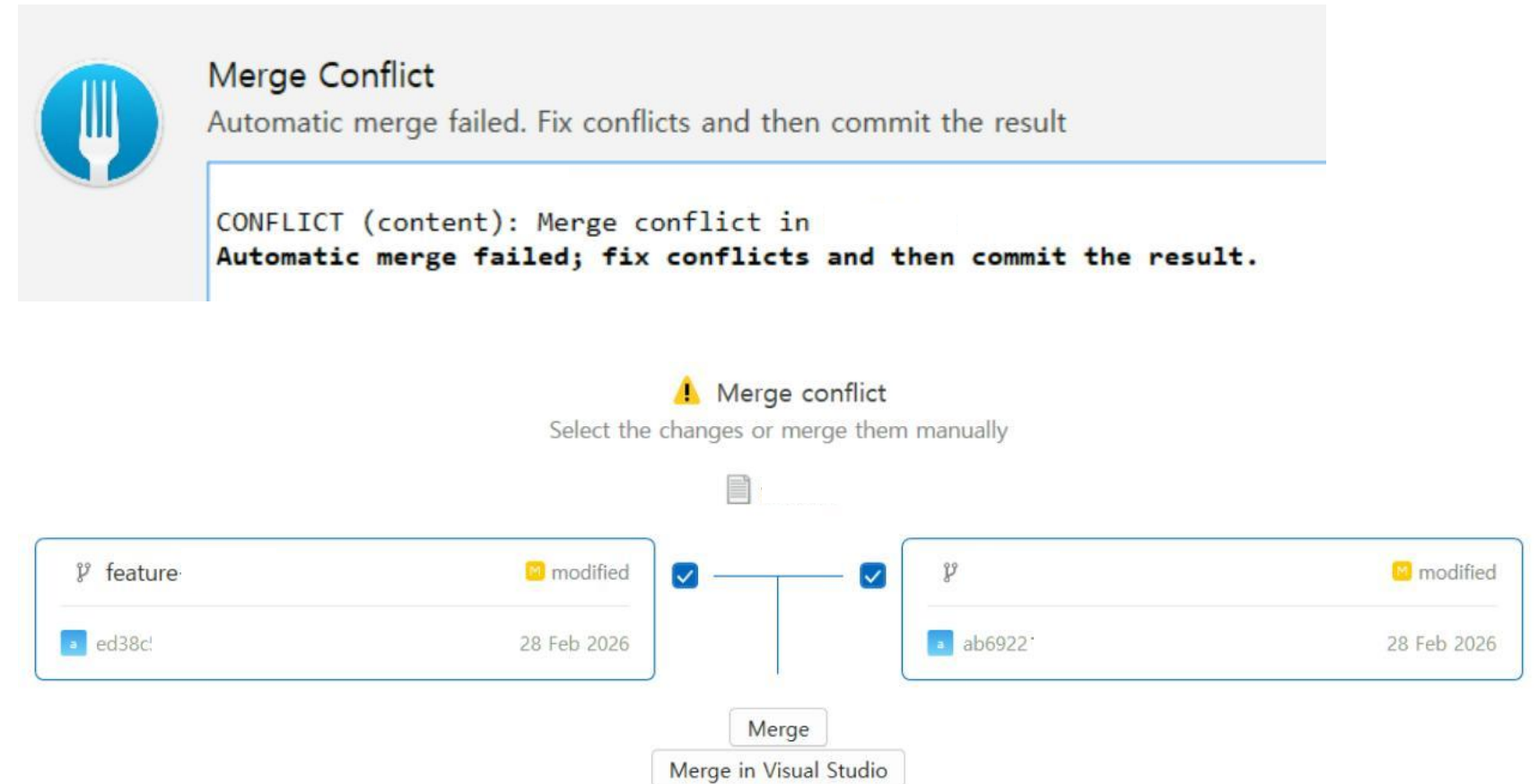
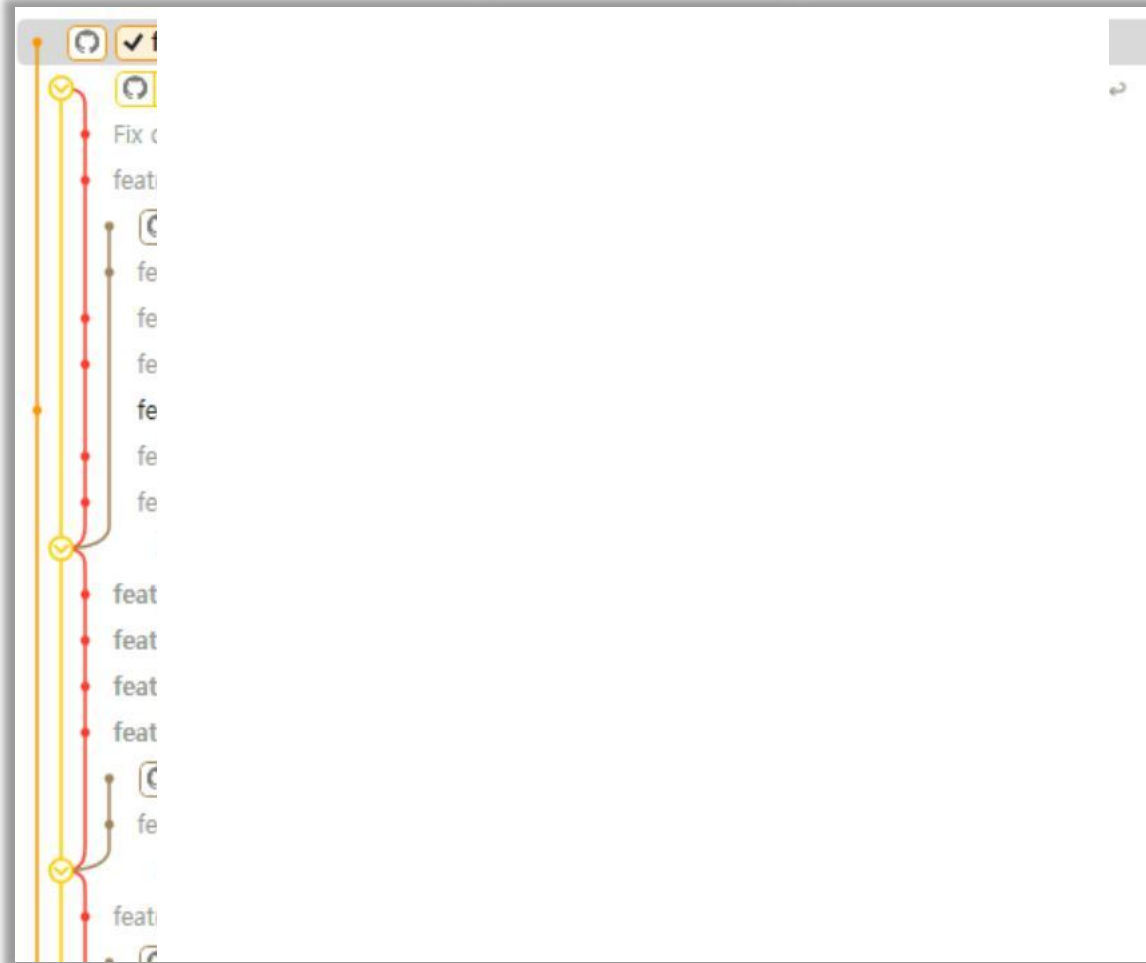
참조 2개
int GetDistance(ANode a, ANode b)
{
    return 10 * (Mathf.Abs(a.indexX - b.indexX) + Mathf.Abs(a.indexY - b.indexY));
}
```

- A* 알고리즘은 휴리스틱(Heuristic, H) 함수를 사용하여 탐색 범위를 목표 방향으로 좁힐 수 있어 게임 내 길찾기에 적합했습니다.

- 작업자 캐릭터는 2D 그리드 맵에서 상하좌우 이동만 허용되었기 때문에 '맨해튼 거리' 휴리스틱을 사용하였습니다.

Fork : 대규모 팀 협업을 위한 버전 관리 경험

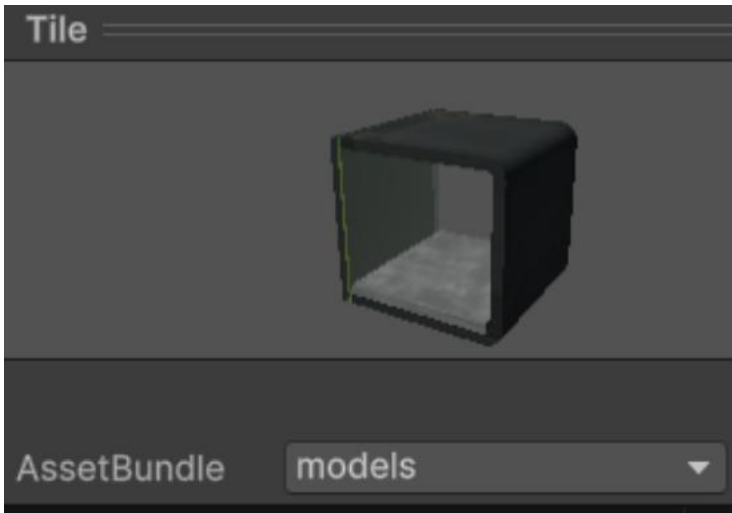
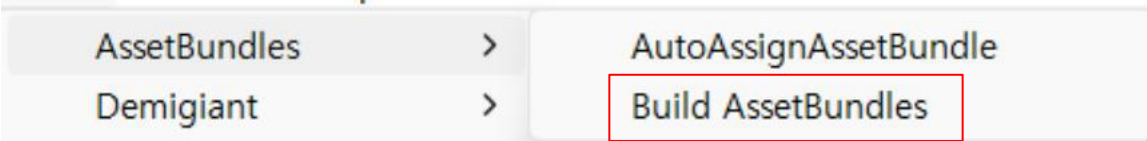
13명이라는 많은 인원이 동시에 개발을 진행하는 환경에서 코드 충돌은 피할 수 없는 문제였습니다.
이에 대비하고자 GUI 클라이언트인 'Fork'를 Github와 연동하여 전체 커밋 분기 및 작업 내역을 관리하였습니다.



- 작업 내역 간 충돌 발생 시 수정된 코드의 전후 관계를 파악해 병합하였습니다.
 - 리버트(Revert) 기능을 통해 이전 커밋 내역으로 변경사항이 복구되는 것을 확인하고 변경사항에 대한 충돌을 해결할 수 있었습니다.
 - 리베이스(Rebase) 기능으로 전체 팀의 공용 main 분기와 개별 팀이 사용하는 다수의 분기들을 하나의 흐름으로 통합하였습니다.
- 그 덕분에 로컬 분기들과 main 분기의 버전 관리를 용이하게 할 수 있게 되었습니다.

에셋 번들(Asset Bundle) : 런타임 자원 관리를 위한 리소스 패키징

프로젝트 내에서 동적으로 생성되는 프리팹과 개별 모델링, GUI 등의 에셋들을 에셋 번들로 구조화하여 관리하였습니다.



project-lup / projectlup-v2025-cdn Public	
AssetBundles	patch 2025-02-21
README.md	patch 2025-12-20
Versions.json	patch 2025-02-21
audios	patch 2025-01-10
data	patch 2025-02-21
gui	patch 2025-12-20
image	patch 2025-02-21
models	patch 2025-02-21
shaders	patch 2025-01-10 (2)
vfx	patch 2025-02-21
videos	patch 2025-12-20

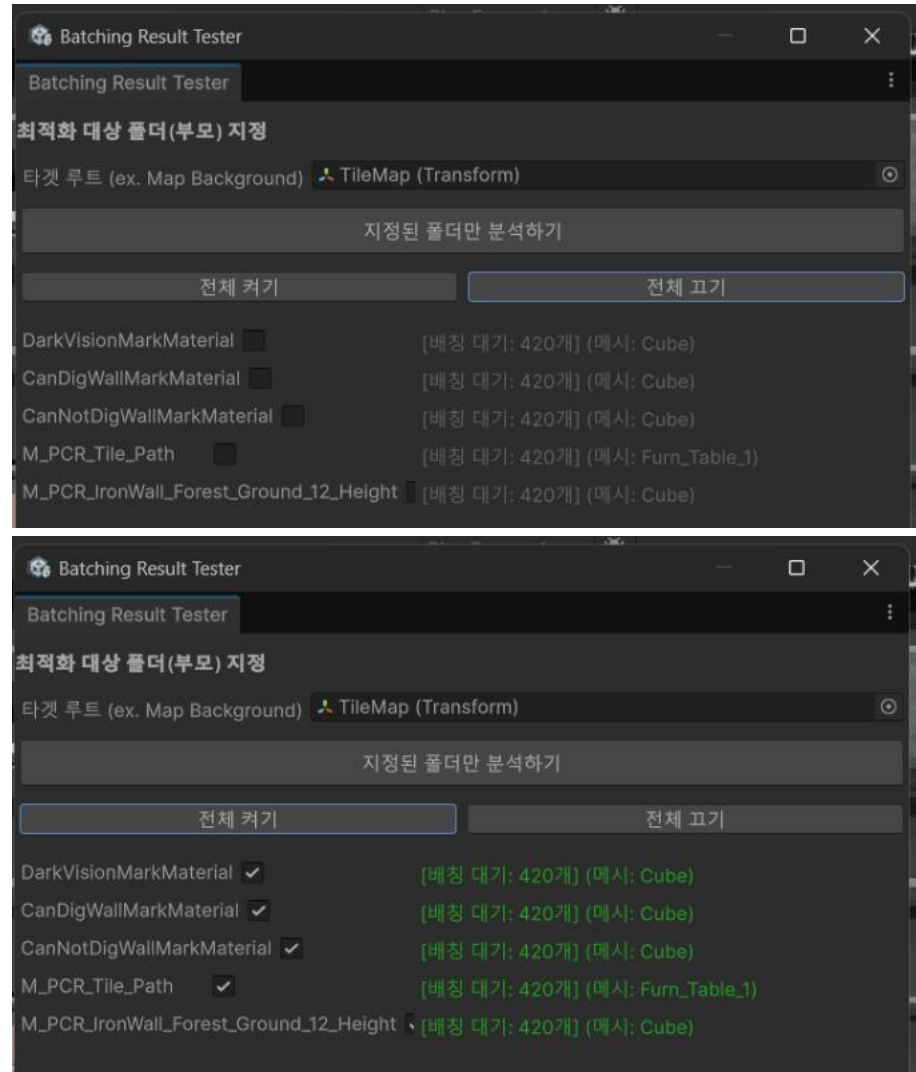
- 게임 진행 상황에 맞춰 필요한 시점에만 자원을 호출하는 런타임 동적 로딩을 위해 에셋 번들 방식을 적용하였습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

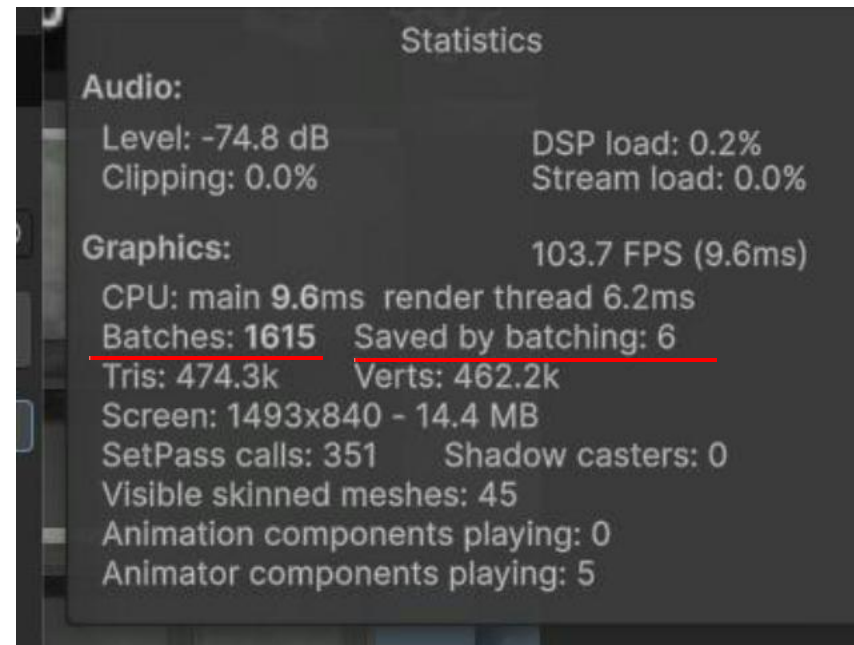
GPU Instancing : 통합 렌더링을 통한 Draw Call 40% 최적화

맵 내 배치된 그리드맵(타일맵)은 같은 규격의 개체가 일정 간격으로 반복 배치되어 있는 개체 그룹입니다.

GPU Instancing 을 통해 같은 Material과 Mesh를 가진 개체에 들어가는 렌더링 연산을 통합 처리하는 에디터 툴을 작성하였습니다.



<Before>



<After>



(Github) GPUOptimizationTester.cs

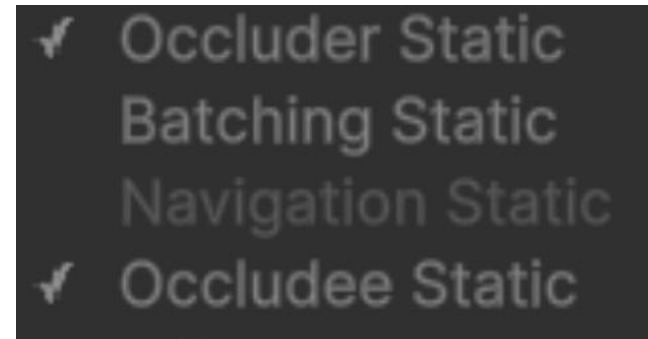
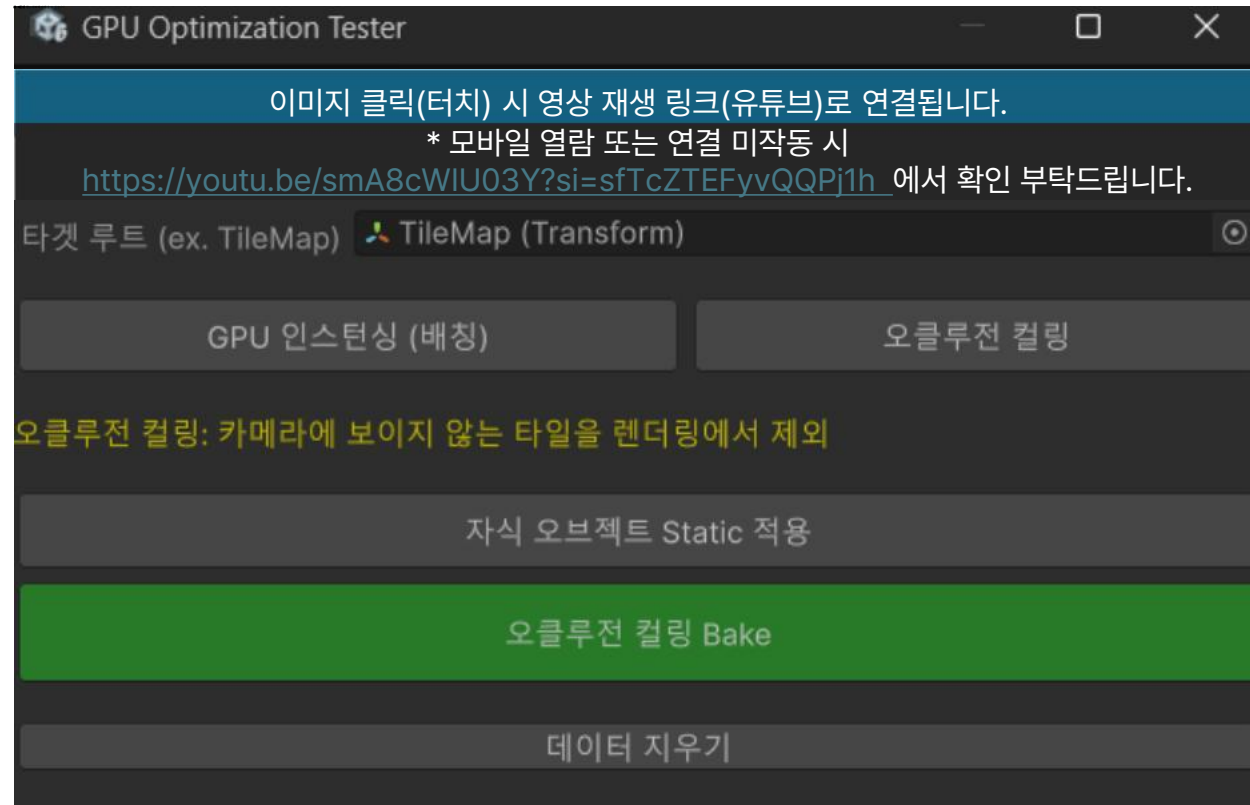
[projectlup-v2025/Assets/8_Editor/Editor/GPUOptimizationTester.cs](https://github.com/projectlup/projectlup-v2025/Assets/8_Editor/Editor/GPUOptimizationTester.cs) at feature/PCR/PSE_Current · project-lup/projectlup-v2025

- GPU Instancing 을 적용한 덕분에 Batches : 1615 -> 961 로 Draw Call 이 약 40.5% 감소되는 효과를 얻을 수 있었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

오클루전 컬링(Occlusion Culling) – 불필요한 GPU 연산 방지

다른 오브젝트에 가려진 오브젝트들을 렌더링하지 않게 오클루전 컬링(Occlusion Culling)을 적용하였습니다.



Occluder : 다른 오브젝트를 가리거나 가려질 오브젝트

Occludere : Occluder 에 의해 가려질 오브젝트

```
StaticEditorFlags flags = GameObjectUtility.GetStaticEditorFlags(renderer.gameObject);  
flags |= StaticEditorFlags.OccluderStatic;  
flags |= StaticEditorFlags.OccludeeStatic;
```

```
GameObjectUtility.SetStaticEditorFlags(renderer.gameObject, flags);  
count++;
```

(Github) GPUOptimizationTester.cs (138번줄 ~ 144번줄)

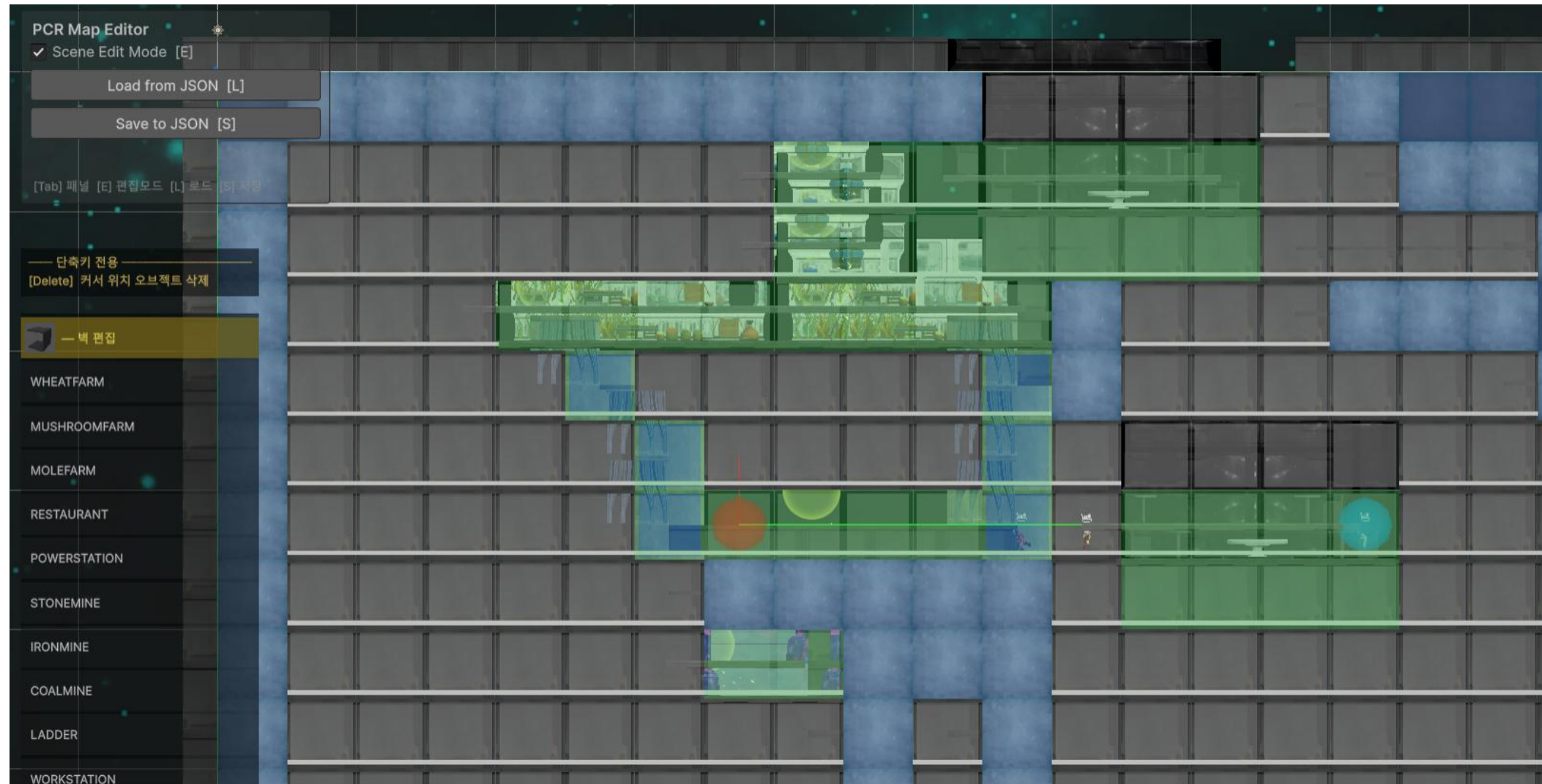
[projectlup-v2025/Assets/8_Editor/Editor/GPUOptimizationTester.cs](https://github.com/project-lup/projectlup-v2025/blob/main/Assets/8_Editor/Editor/GPUOptimizationTester.cs) at feature/PCR/PSE_Current · project-lup/projectlup-v2025

- 오클루전 컬링 데이터를 Bake하려면 게임 내에서 움직이지 않는 오브젝트들(타일 맵 등)에 Static 설정 적용이 필요합니다.
- 오클루전 컬링을 자동으로 적용하기 위한 에디터 툴을 제작하여 오클루전 컬링 적용 효과를 확인하기 위해 수십 개 이상의 오브젝트들을 수동으로 적용하는 수고를 덜 수 있었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

커스텀 맵 에디터 – JSON 데이터 수정 편의를 위한 툴 제작

씬 뷰에서 게임 뷰와 똑같이 렌더링된 모습의 맵을 보면서 맵 데이터(JSON)를 수정할 수 있는 맵 에디터를 제작하였습니다.



(Github) MapEditorWindow.cs

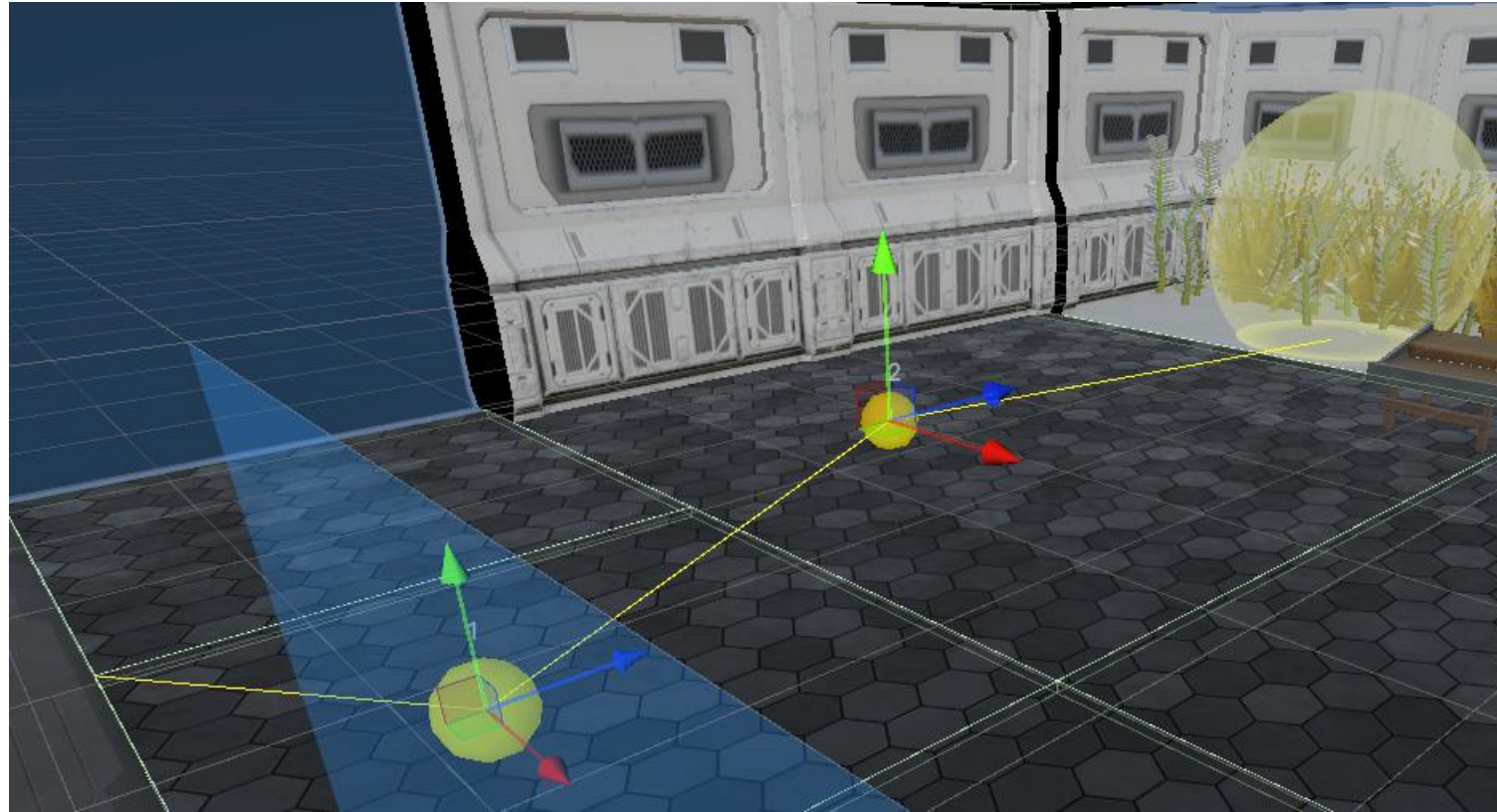
https://github.com/a-i-am/projectlup-v2025-personal/blob/main/Assets/8_Editor/Editor/MapEditorWindow.cs

- 설치한 맵 내 요소를 삭제/편집하는 기능이 미구현된 상태에서 빠르게 맵을 디자인하고 실제 게임에 반영되도록 하였습니다.
- 기존에 JSON 런타임 텍스트 데이터를 일일이 내용을 훑어 수정해야 했던 불편사항을 해결하였습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

작업자 이동 경로 에디터 - 건물 내부 이동 경로를 커스텀

작업자 캐릭터가 건물 바깥에서 a* 로직으로 이동하는 로직이 종료된 후에, 미리 설정된 pathpoint로 건물 내부를 정해진 경로대로 이동할 수 있도록 하였습니다.



경로 편집 도구

Way Point 추가

마지막 포인트 삭제

Add Component

```
EditorGUI.BeginChangeCheck();

for (int i = 0; i < structure.localWaypoints.Count; i++)
{
    Vector3 worldPos = structure.transform.TransformPoint(structure.localWaypoints[i]);

    Vector3 newWorldPos = Handles.PositionHandle(worldPos, Quaternion.identity);

    Handles.Label(newWorldPos + Vector3.up * 0.5f, $"{i + 1}");

    if (EditorGUI.EndChangeCheck())
    {
        Undo.RecordObject(structure, "Move Waypoint");
        structure.localWaypoints[i] = structure.transform.InverseTransformPoint(newWorldPos);
    }
}
```

(Github) PCRStructureEditor.cs (18번줄 ~ 38번줄)

https://github.com/a-i-am/projectlup-v2025-personal/blob/main/Assets/8_Editor/Editor/PCRStructureEditor.cs

- 인스펙터 창에서 건물 내부에서 캐릭터가 이동할 지점을 직접 편집 및 생성/삭제할 수 있는 에디터를 제작하였습니다.
- 이동 경로를 편집 및 추가할 때 실제 오브젝트를 하이어라키에 생성하지 않아도 오브젝트처럼 직접 움직여 경로가 될 위치를 눈으로 확인할 수 있습니다.